

Best Time to Buy and Sell Stock II

A Complete Dynamic Programming Walkthrough

Recursion → Memoization → Tabulation → Space Optimization → Four-Variable $O(1)$ Solution

What this document covers

- Understanding the problem and why it is a recursion problem
- Identifying the recursive state — (index, buy)
- Building the full recursive solution step by step
- Converting recursion into Bottom-Up Tabulation with a full dry run
- Space optimization from an $O(N)$ table to two rolling arrays
- The final four-variable $O(1)$ space solution

1. Problem Statement

You are given an array **prices**, where **prices[i]** is the price of the stock on day *i*.

```
prices = [7, 1, 5, 3, 6, 4]
```

Day	0	1	2	3	4	5
Price	7	1	5	3	6	4

Rules

You may perform **multiple transactions**. The sequence Buy → Sell → Buy → Sell → Buy → Sell is allowed. However, Buy → Buy → Sell is **not** allowed — you can hold only one stock at a time.

Example

Best strategy for the array above:

- Buy at 1, Sell at 5 → Profit = 4
- Buy at 3, Sell at 6 → Profit = 3

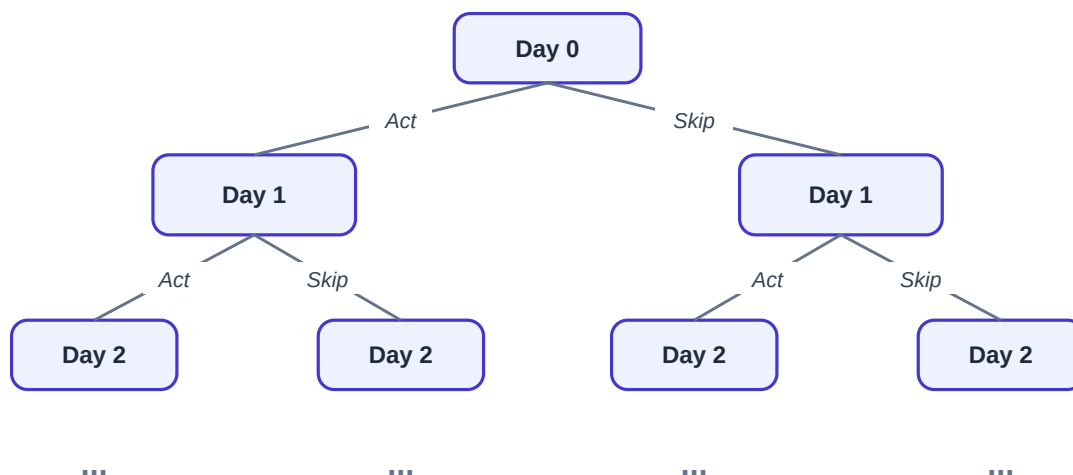
Total profit = 7

Goal

Find the **maximum possible profit** given the buy/sell constraint above.

2. How Should We Think About This?

Forget DP for a moment. Suppose today is Day 3 — you have only two choices: either **do something** (buy or sell) or **skip today**. Every day is exactly like that, and every day branches into more possibilities.



Every day branches into 2 choices $\rightarrow 2 \times 2 \times 2 \times \dots$ possibilities

Each day forks into two choices; over N days this produces roughly 2^N possibilities — the hallmark of a recursion problem.

Part 1 · Recursion (Top-Down)

Step 1 · Understand the Problem

With **prices** = [7,1,5,3,6,4], we can go Buy → Sell → Buy → Sell, but never Buy → Buy, because we can only hold one stock. Our goal is **Maximum Profit**.

Step 2 · Why Recursion?

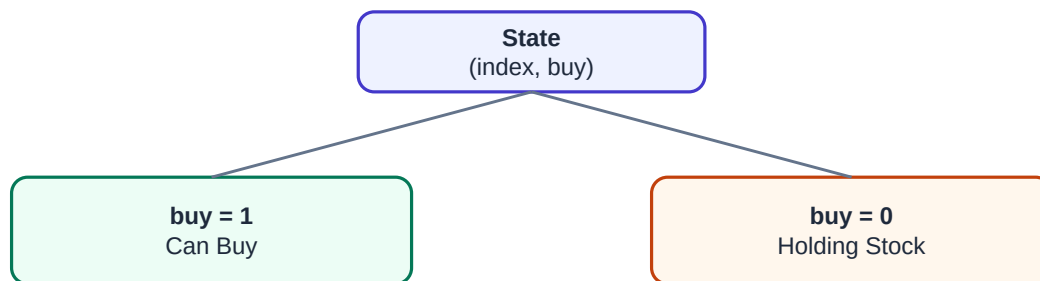
Whenever you see “find the maximum by trying every possibility,” your brain should immediately think **Recursion** — because recursion naturally explores every possible choice. Every decision creates another decision; that becomes a **Decision Tree**.

Step 3 · What Information Defines Our State?

This is the most important step in DP. Whenever you write a recursion, ask: “*What information do I need to know to make my next decision?*”

Information 1 — Current Day. Without knowing the day we cannot know today's stock price, so **index** is necessary.

Information 2 — Am I allowed to buy? This is represented by a flag **buy**: **buy = 1** means we don't own any stock and can Buy; **buy = 0** means we already own a stock and can only Sell. So our recursive state becomes **(index, buy)**. For example, (2,1) means: Day 2, no stock owned, can Buy. And (2,0) means: Day 2, already holding stock, can Sell.



Only two possible states exist for every index

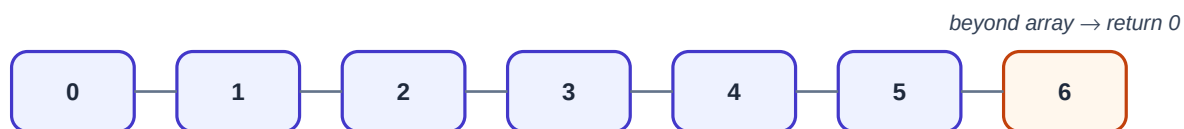
Step 4 · Define the Recursive Function

```
int solve(int index, int buy, vector<int>& prices)
```

This function returns the **maximum profit** starting from Day = index with the current state = buy. So solve(3,1) means: “Starting from Day 3, I don't own any stock — what is the maximum profit I can make?”

Step 5 · Base Case

When `index == n` we have crossed all days: we can't buy and we can't sell, so profit = 0.

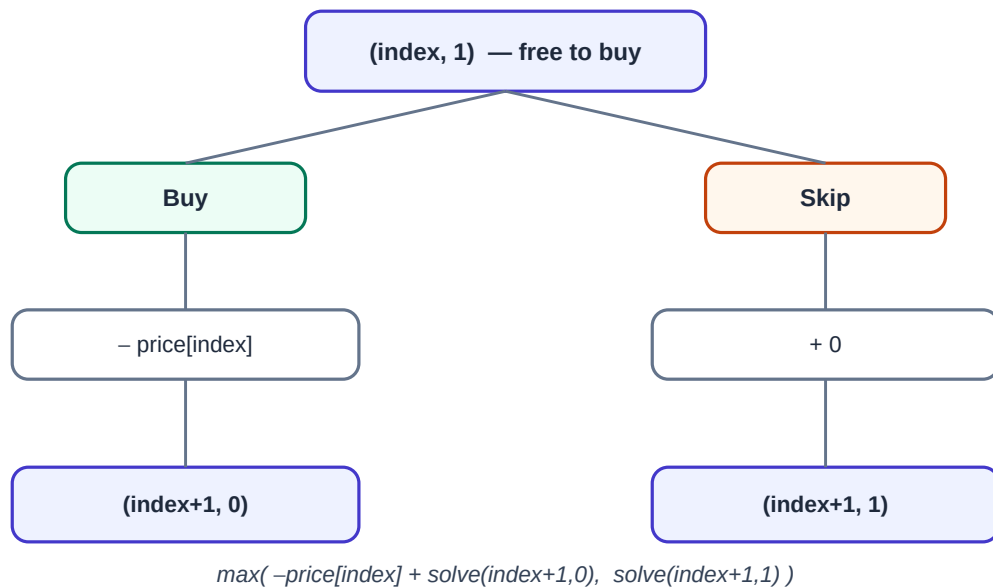


index == prices.size() → no days left → profit = 0

```
if (index == prices.size())  
    return 0;
```

Step 6 · Understanding buy = 1 (Free to Buy)

Choice 1 — Buy: spend $\text{prices}[\text{index}]$ today (profit decreases by that amount), and tomorrow you own stock, so the state becomes $(\text{index}+1, 0)$. **Choice 2 — Skip:** don't buy; tomorrow you still don't own stock, so the state stays $(\text{index}+1, 1)$. We take whichever choice gives the larger profit.

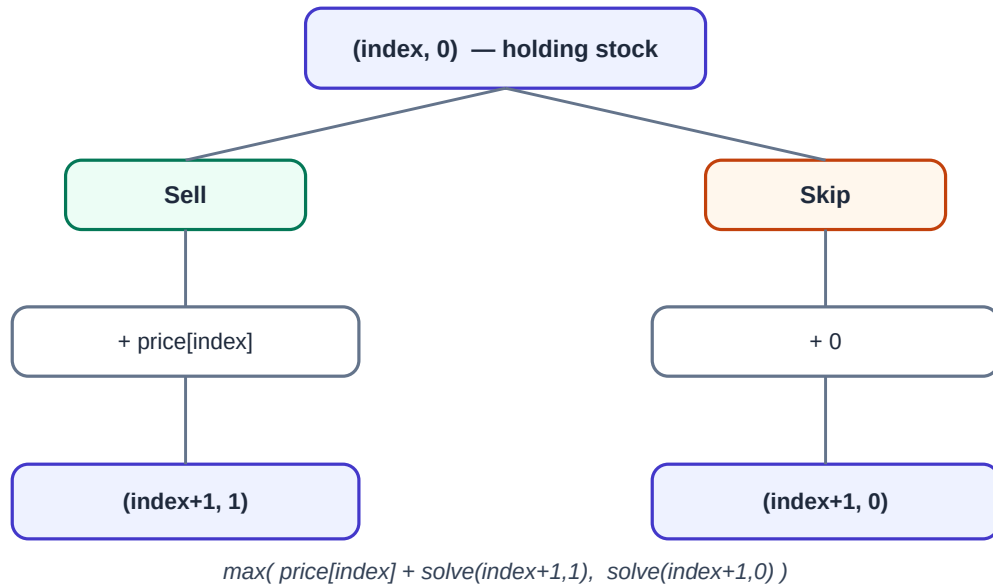


```

max(
    -prices[index] + solve(index + 1, 0),
    solve(index + 1, 1)
)
  
```

Step 7 · Understanding buy = 0 (Holding Stock)

Buying is impossible here. **Choice 1 — Sell:** receive $\text{prices}[\text{index}]$ today; tomorrow you no longer own stock, so the state becomes $(\text{index}+1, 1)$. **Choice 2 — Skip:** don't sell; tomorrow you're still holding stock, so the state stays $(\text{index}+1, 0)$. Again we take the maximum.



```
max(  
    prices[index] + solve(index + 1, 1),  
    solve(index + 1, 0)  
)
```

Step 8 · Complete Recurrence

Combining both cases: if `buy==1`, choose the maximum of Buy or Skip; if `buy==0`, choose the maximum of Sell or Skip.

Step 9 · Complete Recursive Code

```
class Solution {
public:
    int solve(int index, int buy, vector<int>& prices)
    {
        // Base Case
        if (index == prices.size())
            return 0;

        // Can Buy
        if (buy)
        {
            return max(
                -prices[index] + solve(index + 1, 0, prices),
                solve(index + 1, 1, prices)
            );
        }

        // Holding Stock
        return max(
            prices[index] + solve(index + 1, 1, prices),
            solve(index + 1, 0, prices)
        );
    }

    int maxProfit(vector<int>& prices)
    {
        return solve(0, 1, prices);
    }
};
```

Step 10 · Explain Every Line

`if(index==prices.size()) return 0;` — No days left, so no profit possible.

`if(buy)` — Currently not holding stock — we're allowed to Buy.

`-price[index]` — Buying costs money, hence negative.

`solve(index+1,0)` — Tomorrow you own stock, so `buy = 0`.

`solve(index+1,1)` — Skipped today — still free to buy tomorrow.

`price[index]` — Selling earns money, hence positive.

`solve(index+1,1) (after sell)` — After selling, you can buy again.

`solve(index+1,0) (skip sell)` — Skipped selling — still holding stock.

Step 11 · Why `max()`?

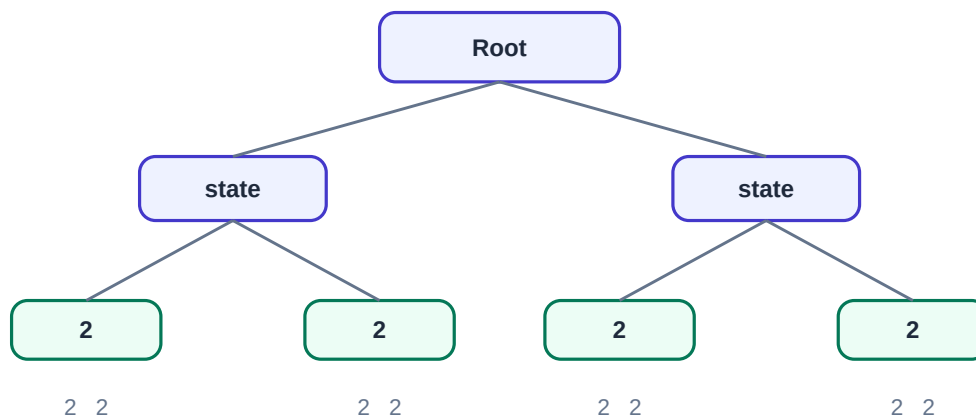
If Buy gives profit 5 and Skip gives profit 8, we obviously choose 8. Therefore every decision is **max(option1, option2)**.

Step 12 · Why Start With solve(0, 1)?

At the beginning we are on Day 0 and do not own any stock, so we are allowed to buy. The initial call solve(0, 1, prices) asks: “Starting from day 0, with the freedom to buy, what is the maximum profit I can make?”

Step 13 · Why Is This Exponential?

Every state explores two choices, and each of those again explores two choices — the recursion tree keeps doubling.



The tree keeps doubling $\approx 2^N$ total calls $\rightarrow O(2^N)$ time

Key Takeaway

The hardest part of this problem is not writing the code — it's identifying the correct state (**index, buy**). Once identified, the recurrence almost writes itself: if you can buy, choose between Buy or Skip; if you're holding a stock, choose between Sell or Skip; return the maximum of the available choices. Once the recursive solution is clear, converting it to Memoization, Tabulation, and finally the $O(1)$ four-variable solution becomes a straightforward optimization rather than a new algorithm.

Time Complexity: $O(2^N)$ · **Space Complexity:** $O(N)$ (recursion stack depth)

Part 3 · Tabulation (Bottom-Up DP)

Step 1 · Start From the Recursive Solution

Every state depends only on **index + 1** — never index+2 or beyond. This single observation tells us exactly how to build the DP table.

Step 2 · What Does `dp[index][buy]` Mean?

Never create a DP table before deciding its meaning. Here, **`dp[index][buy]`** means: “Maximum profit starting from day = index when the current buying state is buy.” For example, `dp[3][1]` means: start from Day 3, not holding any stock — maximum profit? `dp[3][0]` means: start from Day 3, already holding a stock — maximum profit? The DP table stores exactly the same thing as the recursion — nothing changes.

Step 3 · DP Table Size

With $n = 6$ days, the recursive base case accesses `solve(6, ...)`, so the DP table must contain row 6:

```
vector<vector<int>>> dp(n + 1, vector<int>(2, 0));
```

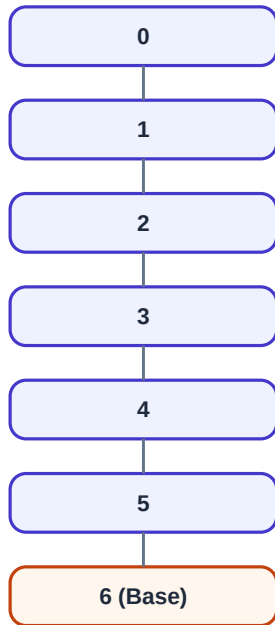
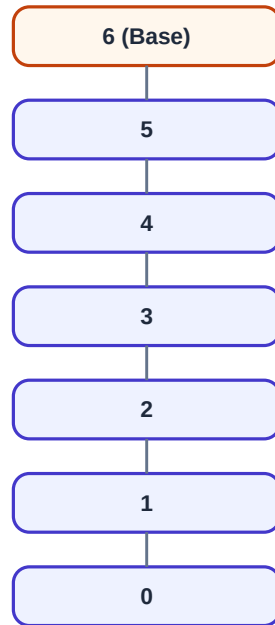
Step 4 · Why Is the Last Row Zero?

Recursion said: no days left → no buying → no selling → profit = 0. So `dp[6][0] = 0` and `dp[6][1] = 0` — the base case is already filled by initializing the table to zero.

Step 5 · Which Direction Should We Fill?

`solve(index)` depends on `solve(index+1)` — we cannot calculate Day 3 before Day 4, because Day 3 needs Day 4. Therefore we must calculate the **last day first** and move toward Day 0. This is why it's called Bottom-Up DP.

```
for (int index = n - 1; index >= 0; index--)
```

Recursion (Top-Down)*Tabulation (Bottom-Up)*

Step 6 · Why Another Loop for buy?

For every day we have two states, buy = 0 and buy = 1, and both must be calculated:

```
for (int buy = 0; buy <= 1; buy++)
```

Step 7 · Translating the Recursion

This is the easiest part — simply replace solve(index+1, ...) with dp[index+1][...]. Nothing else changes.

Complete Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {

        int n = prices.size();
        vector<vector<int>> dp(n + 1, vector<int>(2, 0));

        for (int index = n - 1; index >= 0; index--)
        {
            for (int buy = 0; buy <= 1; buy++)
            {
                if (buy)
                {
                    dp[index][buy] = max(
                        -prices[index] + dp[index + 1][0],
                        dp[index + 1][1]
                    );
                }
                else
                {
                    dp[index][buy] = max(
                        prices[index] + dp[index + 1][1],
                        dp[index + 1][0]
                    );
                }
            }
        }

        return dp[0][1];
    }
};
```

Return dp[0][1]: at Day 0 we are not holding stock and are allowed to buy — exactly the state (index=0, buy=1).

Complete Dry Run

Input: prices = [7, 1, 5, 3, 6, 4], n = 6. All entries start at $dp[6][0] = dp[6][1] = 0$.

index = 5 (price = 4)		
	Option	Value
buy = 0 (Sell)	$4 + dp[6][1]$	4
buy = 0 (Skip)	$dp[index+1][0]$	0
→ $dp[index][0]$	$\max(\dots)$	4
buy = 1 (Buy)	$-4 + dp[6][0]$	-4
buy = 1 (Skip)	$dp[index+1][1]$	0
→ $dp[index][1]$	$\max(\dots)$	0

index = 4 (price = 6)		
	Option	Value
buy = 0 (Sell)	$6 + dp[5][1]$	6
buy = 0 (Skip)	$dp[index+1][0]$	4
→ $dp[index][0]$	$\max(\dots)$	6
buy = 1 (Buy)	$-6 + dp[5][0]$	-2
buy = 1 (Skip)	$dp[index+1][1]$	0
→ $dp[index][1]$	$\max(\dots)$	0

index = 3 (price = 3)		
	Option	Value
buy = 0 (Sell)	$3 + dp[4][1]$	3
buy = 0 (Skip)	$dp[index+1][0]$	6
→ $dp[index][0]$	$\max(\dots)$	6
buy = 1 (Buy)	$-3 + dp[4][0]$	3
buy = 1 (Skip)	$dp[index+1][1]$	0
→ $dp[index][1]$	$\max(\dots)$	3

index = 2 (price = 5)		
	Option	Value
buy = 0 (Sell)	$5 + dp[3][1]$	8
buy = 0 (Skip)	$dp[index+1][0]$	6

→ dp[index][0]	max(...)	8
buy = 1 (Buy)	-5 + dp[3][0]	1
buy = 1 (Skip)	dp[index+1][1]	3
→ dp[index][1]	max(...)	3

index = 1 (price = 1)		
	Option	Value
buy = 0 (Sell)	1 + dp[2][1]	4
buy = 0 (Skip)	dp[index+1][0]	8
→ dp[index][0]	max(...)	8
buy = 1 (Buy)	-1 + dp[2][0]	7
buy = 1 (Skip)	dp[index+1][1]	3
→ dp[index][1]	max(...)	7

index = 0 (price = 7)		
	Option	Value
buy = 0 (Sell)	7 + dp[1][1]	14
buy = 0 (Skip)	dp[index+1][0]	8
→ dp[index][0]	max(...)	14
buy = 1 (Buy)	-7 + dp[1][0]	1
buy = 1 (Skip)	dp[index+1][1]	7
→ dp[index][1]	max(...)	7

A Common Doubt — Why Can dp[0][0] Become 14?

dp[index][0] assumes you're already holding a stock when you arrive at that day; it does not account for the cost of buying that stock earlier. At index = 0, selling immediately gives +7, and later the DP can earn another +7, totaling 14. This is an internal DP state, not a valid starting state — the initial purchase cost is handled in a previous buy = 1 state through the -prices[index] transition. That's why the algorithm correctly returns **dp[0][1]**, not dp[0][0].

Final DP Table

Index	buy = 0 (Holding Stock)	buy = 1 (Can Buy)
0	14	7
1	8	7

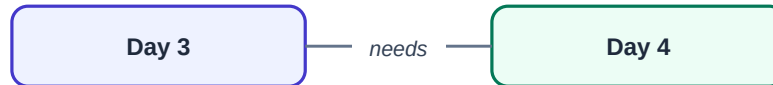
Index	buy = 0 (Holding Stock)	buy = 1 (Can Buy)
2	8	3
3	6	3
4	6	0
5	4	0
6	0	0

The answer is **dp[0][1] = 7**, because we start on day 0 with no stock in hand, meaning we are allowed to buy.

Part 4 · Space Optimization

Step 1–2 · What Does One State Actually Need?

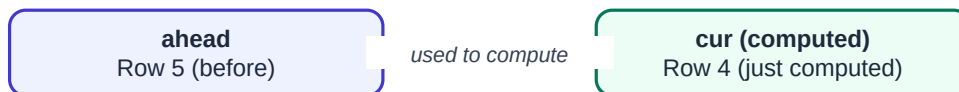
Looking only at the dependencies: $dp[index][1]$ uses $dp[index+1][0]$ and $dp[index+1][1]$. $dp[index][0]$ uses exactly the same two values. Neither state ever needs $dp[index+2]$ or beyond — only the **next row**.



Day 3 cannot be computed until Day 4 is known

Step 3–4 · Replace Rows With Two Arrays

At any moment we only need the **current row** and the **next row**. Call $dp[index+1]$ **ahead**, and $dp[index]$ **cur**. Once a row is finished it becomes tomorrow's row: **ahead = cur**.



ahead = cur → becomes next iteration's ahead

After computing Day 4, ahead is updated to hold Day 4's values

Step 5 · Array Sizes

Each row only has two values — $buy=0$ and $buy=1$ — so both **ahead** and **cur** are simply vectors of size 2:

```
vector<int> ahead(2, 0);
vector<int> cur(2, 0);
```

Step 6 · Translating the Recursion

$dp[index][buy]$ becomes $cur[buy]$; $dp[index+1][buy]$ becomes $ahead[buy]$. Everything else stays identical.

Complete Code


```
class Solution {
public:
    int maxProfit(vector<int>& prices) {

        int n = prices.size();
        vector<int> ahead(2, 0);
        vector<int> cur(2, 0);

        for (int index = n - 1; index >= 0; index--)
        {
            for (int buy = 0; buy <= 1; buy++)
            {
                if (buy)
                {
                    cur[buy] = max(-prices[index] + ahead[0], ahead[1]);
                }
                else
                {
                    cur[buy] = max(prices[index] + ahead[1], ahead[0]);
                }
            }
            ahead = cur;
        }

        return ahead[1];
    }
};
```

Key Intuition

Think of **ahead** and **cur** as two snapshots of the DP table: Day $i+1$ (already computed) is used to compute Day i (currently computing). After finishing Day i , that row becomes the “next day” for the following iteration — **ahead** = **cur**. Nothing magical happens: each row depends only on the immediately next row, so storing the entire $n \times 2$ table is unnecessary.

Part 5 · Four-Variable Optimization

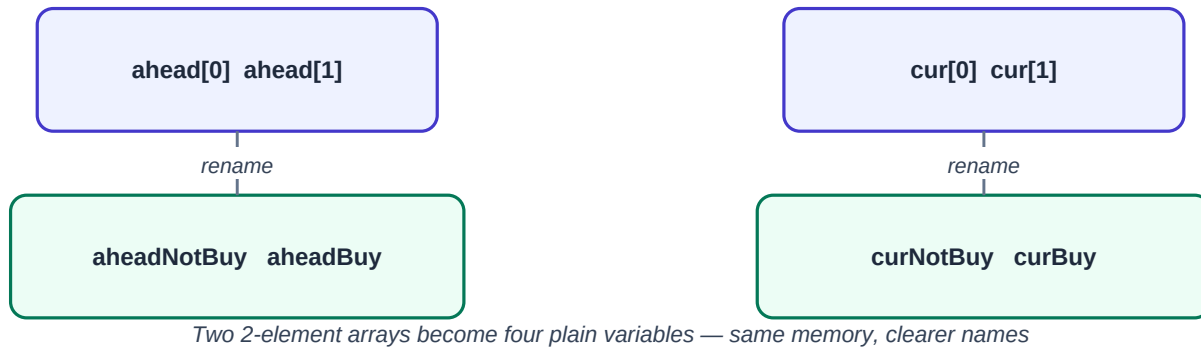
Most people memorize the four-variable version of this code (aheadBuy, aheadNotBuy, curBuy, curNotBuy) without understanding where the variables came from. The truth is: nothing new is happening — we are simply replacing the two size-2 arrays with four named variables.

Step 1–2 · Start From the Space-Optimized Arrays

ahead[0] corresponds to dp[index+1][0], ahead[1] to dp[index+1][1]; cur[0] and cur[1] correspond to dp[index][0] and dp[index][1].

Step 3 · Replace Array Elements With Meaningful Names

Recall that buy=0 means holding stock (cannot buy) and buy=1 means free to buy. So ahead[0] → **aheadNotBuy**, ahead[1] → **aheadBuy**, cur[0] → **curNotBuy**, cur[1] → **curBuy**. Exactly the same memory — just different names.



Step 4 · Convert the Formulas

```
curBuy = max(-prices[index] + aheadNotBuy, aheadBuy);
curNotBuy = max(prices[index] + aheadBuy, aheadNotBuy);
```

Nothing changed except the names.

Step 5 · Updating

Instead of ahead = cur;, we now copy the individual variables:

```
aheadBuy = curBuy;
aheadNotBuy = curNotBuy;
```

Complete Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {

        int n = prices.size();

        int aheadBuy = 0;
        int aheadNotBuy = 0;
        int curBuy = 0;
        int curNotBuy = 0;

        for (int index = n - 1; index >= 0; index--)
        {
            curBuy = max(-prices[index] + aheadNotBuy, aheadBuy);
            curNotBuy = max(prices[index] + aheadBuy, aheadNotBuy);

            aheadBuy = curBuy;
            aheadNotBuy = curNotBuy;
        }

        return aheadBuy;
    }
};
```

Explain Every Line

aheadBuy = 0 — Represents $dp[index+1][1]$ — tomorrow, allowed to buy. Base case = 0.

aheadNotBuy = 0 — Represents $dp[index+1][0]$ — tomorrow, holding stock. Base case = 0.

curBuy — Represents $dp[index][1]$ — current day, allowed to buy.

curNotBuy — Represents $dp[index][0]$ — current day, holding stock.

The Buy Formula

$curBuy = \max(-price[index] + aheadNotBuy, aheadBuy)$. **Option 1 — Buy:** pay the price today; tomorrow you're holding stock, so add aheadNotBuy. **Option 2 — Skip:** do nothing; tomorrow you can still buy, so use aheadBuy. Take the maximum.

The Sell Formula

$curNotBuy = \max(price[index] + aheadBuy, aheadNotBuy)$. **Option 1 — Sell:** receive money today; tomorrow you can buy again, so add aheadBuy. **Option 2 — Skip:** stay holding stock, so use aheadNotBuy. Take the maximum.

Complete Dry Run

Input: prices = [7, 1, 5, 3, 6, 4]. Initially aheadBuy = 0, aheadNotBuy = 0.

index = 5 (price = 4)		
	Option	Value
curBuy ← Buy	-4 + aheadNotBuy	-4
curBuy ← Skip	aheadBuy	0
→ curBuy = max(...)		0
curNotBuy ← Sell	4 + aheadBuy	4
curNotBuy ← Skip	aheadNotBuy	0
→ curNotBuy = max(...)		4

index = 4 (price = 6)		
	Option	Value
curBuy ← Buy	-6 + aheadNotBuy	-2
curBuy ← Skip	aheadBuy	0
→ curBuy = max(...)		0
curNotBuy ← Sell	6 + aheadBuy	6
curNotBuy ← Skip	aheadNotBuy	4
→ curNotBuy = max(...)		6

index = 3 (price = 3)		
	Option	Value
curBuy ← Buy	-3 + aheadNotBuy	3
curBuy ← Skip	aheadBuy	0
→ curBuy = max(...)		3
curNotBuy ← Sell	3 + aheadBuy	3
curNotBuy ← Skip	aheadNotBuy	6
→ curNotBuy = max(...)		6

index = 2 (price = 5)		
	Option	Value
curBuy ← Buy	-5 + aheadNotBuy	1
curBuy ← Skip	aheadBuy	3
→ curBuy = max(...)		3
curNotBuy ← Sell	5 + aheadBuy	8
curNotBuy ← Skip	aheadNotBuy	6
→ curNotBuy = max(...)		8

index = 1 (price = 1)		
	Option	Value
curBuy ← Buy	-1 + aheadNotBuy	7
curBuy ← Skip	aheadBuy	3
→ curBuy = max(...)		7
curNotBuy ← Sell	1 + aheadBuy	4
curNotBuy ← Skip	aheadNotBuy	8
→ curNotBuy = max(...)		8

index = 0 (price = 7)		
	Option	Value
curBuy ← Buy	-7 + aheadNotBuy	1
curBuy ← Skip	aheadBuy	7
→ curBuy = max(...)		7
curNotBuy ← Sell	7 + aheadBuy	14

curNotBuy ← Skip	aheadNotBuy	8
→ curNotBuy = max(...)		14

Final Answer

Return **aheadBuy**. After the last update, aheadBuy represents dp[0][1]: start from Day 0, not holding any stock, allowed to buy — exactly the initial state of the problem. The final answer is **7**.